



UNIVERSIDAD TECNICA
FEDERICO SANTA MARIA



DEPARTAMENTO DE
ELECTRONICA

ELO329: DISEÑO Y PROGRAMACIÓN ORIENTADA A OBJETOS

Tarea 3: Simulador Gráfico en C++ y Qt de EloTelTags y Aplicación Find My

Alumno: Daniel Reyes

Alumno: Marco Duque

Alumno: Victor Santos

Profesor: Agustín J. González V.

Fecha: 30 de junio de 2026

1. Introducción

El presente proyecto consiste en el desarrollo de un simulador gráfico interactivo implementado en C++ y Qt para modelar un sistema de rastreo colaborativo de dispositivos. El objetivo principal es aplicar los conceptos fundamentales de la programación orientada a objetos y, de manera central, el paradigma de la programación conducida por eventos. A través de este enfoque, se simula una red descentralizada en la que dispositivos móviles activos (celulares y tablets) interactúan dinámicamente dentro de un territorio confinado para escanear, detectar y reportar de manera asíncrona la ubicación de objetos de rastreo pasivos (EloTelTags) hacia un servidor centralizado en la nube (ETNube). Este sistema emula detalladamente las funcionalidades de ecosistemas comerciales como Apple Find My, incorporando la visualización de animaciones de radar, alertas sonoras por proximidad y la consulta interactiva del último paradero conocido de los bienes de un usuario mediante ventanas flotantes sincronizadas en tiempo real.

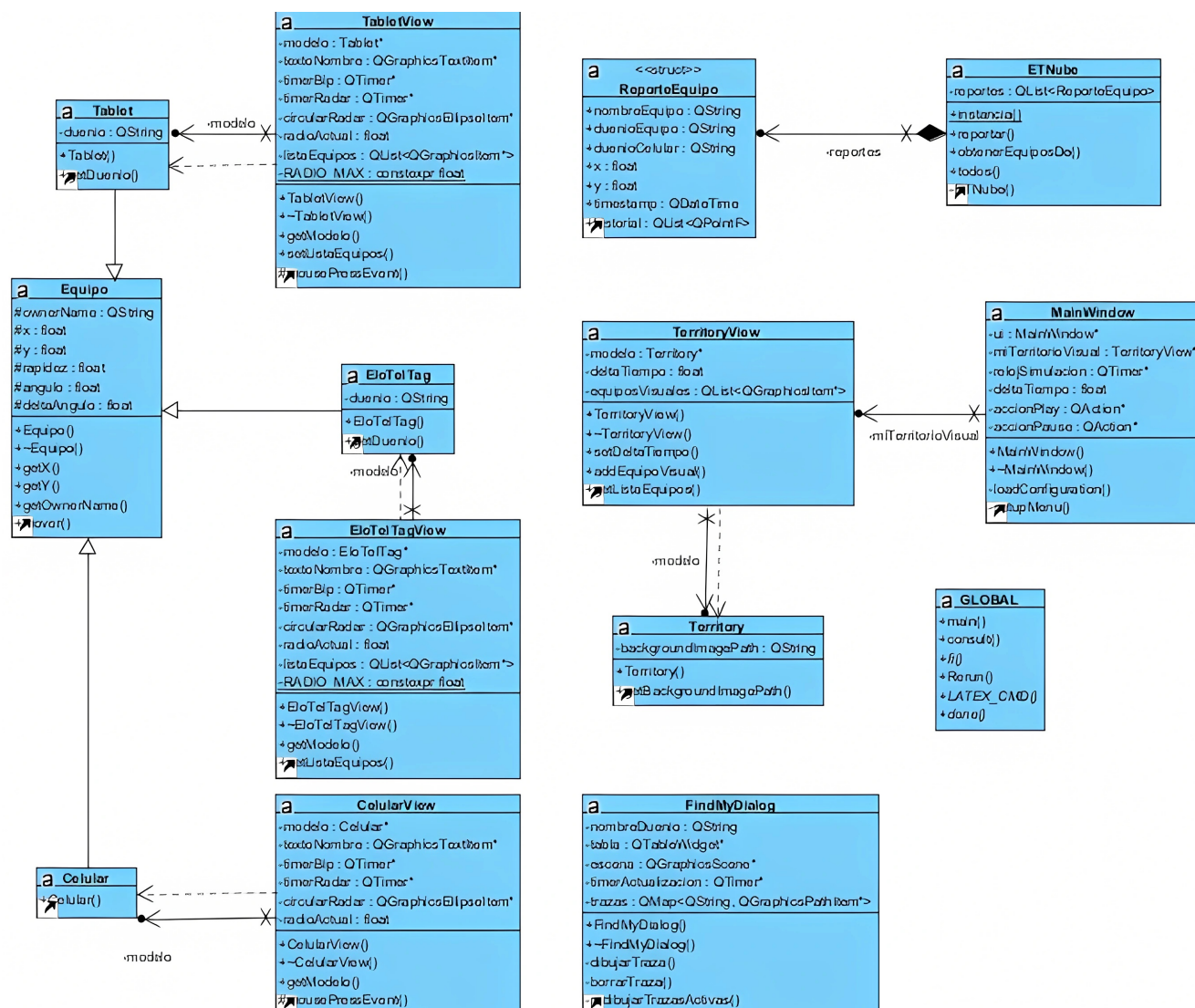
2. Requisitos

- C++.
- Qt Creator
- Archivos de entrada `config.txt` (estado inicial).

3. Diagrama de Clases (UML)

A continuación se presenta el diagrama de clases correspondiente a la Etapa 4 del proyecto, generado a partir de los archivos fuentes finales mediante la herramienta Visual Paradigm.

Las flechas continuas con punta triangular blanca denotan relaciones de herencia (*extends*) apuntando hacia la superclase común **Equipo**. Por otro lado, las líneas punteadas representan dependencias de instanciación, referencias y paso de parámetros entre los componentes lógicos de datos (Modelos) y los componentes visuales de la interfaz de usuario (Vistas).



4. Explicación de la Solución (Interacción en Tiempo de Ejecución)

La arquitectura del software se diseñó bajo el patrón **Modelo-Vista-Controlador (MVC)**, migrado desde Java/JavaFX hacia **C++ con Qt 6** (Qt Widgets y QGraphicsScene). La coordinación central de la aplicación recae sobre la clase **MainWindow**. Al iniciar la ejecución, esta abre un diálogo de selección de archivo, lee los parámetros desde el archivo de configuración e instancia secuencialmente los modelos lógicos (**Celular**, **Tablet**, **EloTelTag**) asociándoles sus respectivas vistas gráficas (**CelularView**, **TableView**, **EloTelTagView**), incorporándolas dentro del contenedor espacial **TerritoryView** (que hereda de **QGraphicsScene**).

Cuando el usuario selecciona la opción *Play* en el menú superior, el flujo de interacciones entre las clases se comporta de la siguiente manera:

- **Ciclo de Movimiento:** **MainWindow** posee un **QTimer** conectado al slot **updateSimulation()** de **TerritoryView**. En cada tick, este método itera sobre la lista interna de equipos visuales, identifica su tipo mediante **dynamic_cast** y llama al método **mover(deltaT, ancho, alto)** del modelo correspondiente, actualizando las coordenadas internas. Luego actualiza la posición visual con **setPos()**.
- **Actualización de la Interfaz:** A diferencia de la implementación JavaFX, Qt no utiliza propiedades reactivas con **bind()**. En cambio, cada tick del **QTimer** de simulación actualiza explícitamente la posición de cada ítem gráfico mediante **setPos()**, lo que Qt redibuja automáticamente en la escena.
- **Efectos de Radar:** Cada vista de equipo (**CelularView**, **EloTelTagView**, **TableView**) posee dos **QTimer** propios: uno que dispara cada *tbip* segundos con un desfase aleatorio inicial (para evitar sincronización de fase), y otro que anima el círculo de radar creciendo 20 pasos en 1 segundo (cada 50 ms) hasta alcanzar un radio de 50 píxeles. El círculo es un **QGraphicsEllipseItem** hijo del ítem principal, creado y destruido dinámicamente en cada pulso. No se reproduce audio en esta versión.
- **Reportes a la Nube:** Al completarse la animación del radar (al llegar a 50 px), **EloTelTagView** y **TableView** calculan la distancia euclidiana hacia cada **CelularView** en la lista compartida de equipos. Si la distancia es menor o igual a 50 píxeles, se reporta a **ETNube** (singleton header-only) mediante **reportar()**, almacenando el nombre del dispositivo, su dueño/a, el dueño/a del celular que lo detectó, la posición del celular en ese instante y acumulando el punto en el historial de posiciones del equipo.
- **Consulta Interactiva (Find My):** Al hacer clic sobre un celular o tablet, el método **mousePressEvent()** captura la interacción y despliega un **QMenu** flotante con las opciones *Find My* y *GFindMy*. Al seleccionar *Find My*, se instancia un **FindMyDialog** que consulta **ETNube** filtrando por dueño/a, muestra una tabla con la última posición conocida de cada equipo y se actualiza automáticamente cada 1 segundo mediante un **QTimer** interno mientras la ventana permanece abierta. Cada fila incluye además un botón de *traza* que dibuja o elimina en la escena principal un **QGraphicsPathItem** con el historial completo de posiciones del equipo.
- **Consulta Gráfica (GFindMy - Extra crédito):** Al seleccionar *GFindMy*, se instancia un **GFindMyDialog** que construye una **QGraphicsScene** propia cargando la misma imagen de fondo de la simulación principal. Sobre ella se dibujan marcadores (círculo naranja + etiqueta) para cada equipo del dueño/a seleccionado/a, actualizando su posición cada 1 segundo desde **ETNube** mediante un **QTimer** interno.

5. Dificultades Presentadas y Soluciones Implementadas

Durante el proceso de elaboración técnica de la tarea, la migración desde Java/JavaFX hacia C++ con Qt 6 enfrentó cuatro desafíos de programación significativos, los cuales se resolvieron mediante la aplicación de conceptos avanzados de orientación a objetos y el uso correcto de las librerías del framework de interfaz:

1. **Dificultad:** *Tipos incompletos al utilizar punteros a `QGraphicsScene` y `QHeaderView` dentro de las vistas.*

Al compilar las clases `EloTelTagView`, `TabletView` y `FindMyDialog`, el compilador g++ (MinGW) arrojaba el error *invalid use of incomplete type* sobre `QGraphicsScene` y `QHeaderView`. Esto ocurría porque las cabeceras incluidas (`QGraphicsEllipseItem`, `QTableWidget`) solo realizan una declaración adelantada (*forward declaration*) de dichas clases, suficiente para almacenar punteros pero no para invocar sus métodos.

Solución Implementada: Se agregaron explícitamente los `#include <QGraphicsScene>` y `#include <QHeaderView>` en los archivos .cpp correspondientes, en lugar de depender de las inclusiones transitivas de Qt. Adicionalmente, se corrigió el uso de `event->screenPos().toPoint()` (válido en Qt5) reemplazándolo por `event->screenPos().toPointF().toPoint()`, ya que en Qt6 `screenPos()` retorna un `QPointF` directamente y el encadenamiento de conversiones requirió ajuste de tipos.

2. **Dificultad:** *Determinar correctamente qué clase es responsable de detectar y reportar la proximidad entre dispositivos.*

En una primera versión, la lógica de detección se ubicó dentro de `CelularView`, asumiendo (por error de interpretación del enunciado) que era el celular quien escaneaba activamente en busca de Tags y Tablets cercanos. Esto generaba comportamientos inconsistentes con el enunciado, que especifica que son los `EloTelTags` y las `Tablets` quienes buscan celulares cada *thip* segundos, reportando el celular únicamente al ser alcanzado por la señal.

Solución Implementada: Se invirtió la responsabilidad: `EloTelTagView` y `TabletView` pasaron a calcular la distancia euclidiana hacia cada `CelularView` de la escena (recibida mediante `setListaEquipos()`) al completar la animación de su propio radar, e invocar `ETNube::reportar()` solo en ese momento. `CelularView` quedó reducido a mostrar su propio radar visual, sin participar en la lógica de detección, manteniendo el principio de responsabilidad única por clase.

3. **Dificultad:** *Inconsistencia entre el nombre del equipo y el nombre de su dueño/a al reportar a `ETNube`.*

Inicialmente, `ETNube::reportar()` solo recibía el nombre del equipo (ej. "maleta") y el dueño/a del celular que lo detectaba, sin registrar explícitamente a quién pertenecía el propio Tag o Tablet. Esto impedía que la ventana `FindMyDialog` de una persona mostrara correctamente sus propios equipos, ya que no existía una forma de filtrar por "dueño/a del accesorio." en lugar de "dueño/a del celular detector".

Solución Implementada: Se agregó el atributo `duenio` a las clases `EloTelTag` y `Tablet` (independiente del nombre propio del equipo), y se amplió la firma de `ETNube::reportar()` a cinco parámetros: nombre del equipo, dueño/a del equipo, dueño/a del celular detector, y posición. `FindMyDialog` pasó a filtrar mediante `obtenerEquiposDe(duenio)` comparando contra el dueño/a del equipo, no del celular.

4. **Dificultad:** *Pérdida del historial de posiciones al implementar la función de traza.*

Al solicitar que cada equipo en `FindMyDialog` contara con un botón para mostrar/ocultar su trayectoria en el mapa, la primera implementación de `ETNube` solo almacenaba la última posición reportada por equipo (sobrescribiéndola en cada llamado a `reportar()`), por lo que no existía información histórica disponible para trazar una línea.

Solución Implementada: Se modificó la estructura `ReporteEquipo` dentro de `ETNube` para incluir un `QList<QPointF>historial`, acumulando cada nueva posición detectada sin descartar las anteriores. `FindMyDialog` construye la traza leyendo directamente ese historial mediante un `QPainterPath`, dibujándolo como un `QGraphicsPathItem` sobre la escena principal (`TerritoryView`) con `setZValue(100)` para garantizar su visibilidad sobre el fondo y los demás equipos. Al cerrar la ventana `FindMyDialog`, su destructor recorre el mapa de trazas activas y las elimina de la escena, evitando que queden líneas huérfanas en el mapa.